

Uber

We have designed and implemented a simulation of a minimal Uber system using Decorator, State, Composite and Iterator design patterns. The base unit of operation is a standard trip which can be dynamically enhanced with the concrete decorators: PriorityTrip and UberBlack; this alters functionality and pricing. The trip's lifecycle is managed using different states, having the trip sequentially transition through RequestedState, DriverAssignedState, InTransitState and CompletedState. The system also allowed for the trip to be cancelled in all states except for CompletedState, where it will inform the user that the ride is completed and if a ride is cancelled in the InTransitState then the passenger will be charged 60% of the cost. Geographically, operational areas are structured using the Composite pattern, organising nested regions into a tree hierarchy. Individual trips and composite regional hubs share a uniform interface. Finally, the system incorporates the Iterator pattern through two iterators: a CompleteHierarchyIterator for a full recursive traversal of the regional composite structure and an ActiveTripIterator that filters specifically for active, in-transit rides.

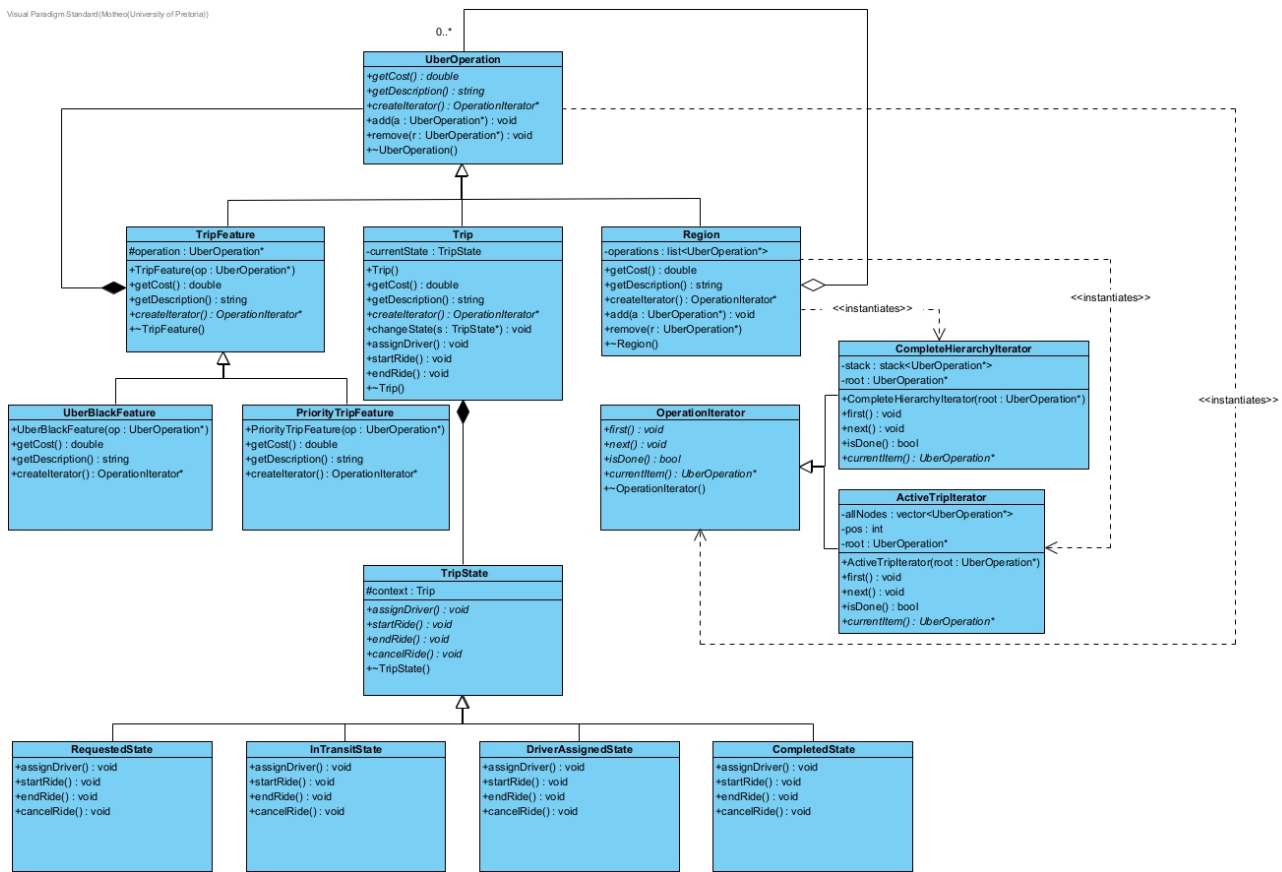
Sudud Abughazaleh (u25282728)
Tshegofatso Mautjane (u25645006)
Motheo Motsemme (u25099184)

Gof participant mapping

- **Composite Pattern:**
 - * Component: UberOperation:- Defines the interface for all operations, including cost calculations and status checks.
 - * Composite: Region- Contains a collection of UberOperation pointers (which can be smaller regions or individual trips)
 - * Leaf: Trip - The individual work item representing a single passenger ride.
- **State Pattern**
 - * Context: Trip - Maintains a pointer to its current state.
 - * State: TripState - The abstract base class defining valid state transitions (e.g., assignDriver(), startRide()).
 - * ConcreteState: RequestedState, InTransitState, CompletedState, CancelledState.
- **Decorator Pattern**
 - * Component: UberOperations
 - * Concrete Component: Trip
 - * Decorator: TripFeature - Holds a pointer to an UberOperation.
 - * ConcreteDecorators: UberBlackFeature, PriorityTripFeature. The modify the cost and description dynamically and can be stacked.
- **Iterator Pattern**
 - * Aggregate: Region.
 - * Iterator: OperationIterator - Abstract base class for traversal
 - * ConcreteIterator:
 - ✦ CompleteHierarchyIterator: Visits every nested region and trip.
 - ✦ ActiveTripIterator: Filters the traversal to only return Trip objects that are currently in the InTransitState.

UML diagram

Visual Paradigm Standard (Mofibo University of Pretoria)



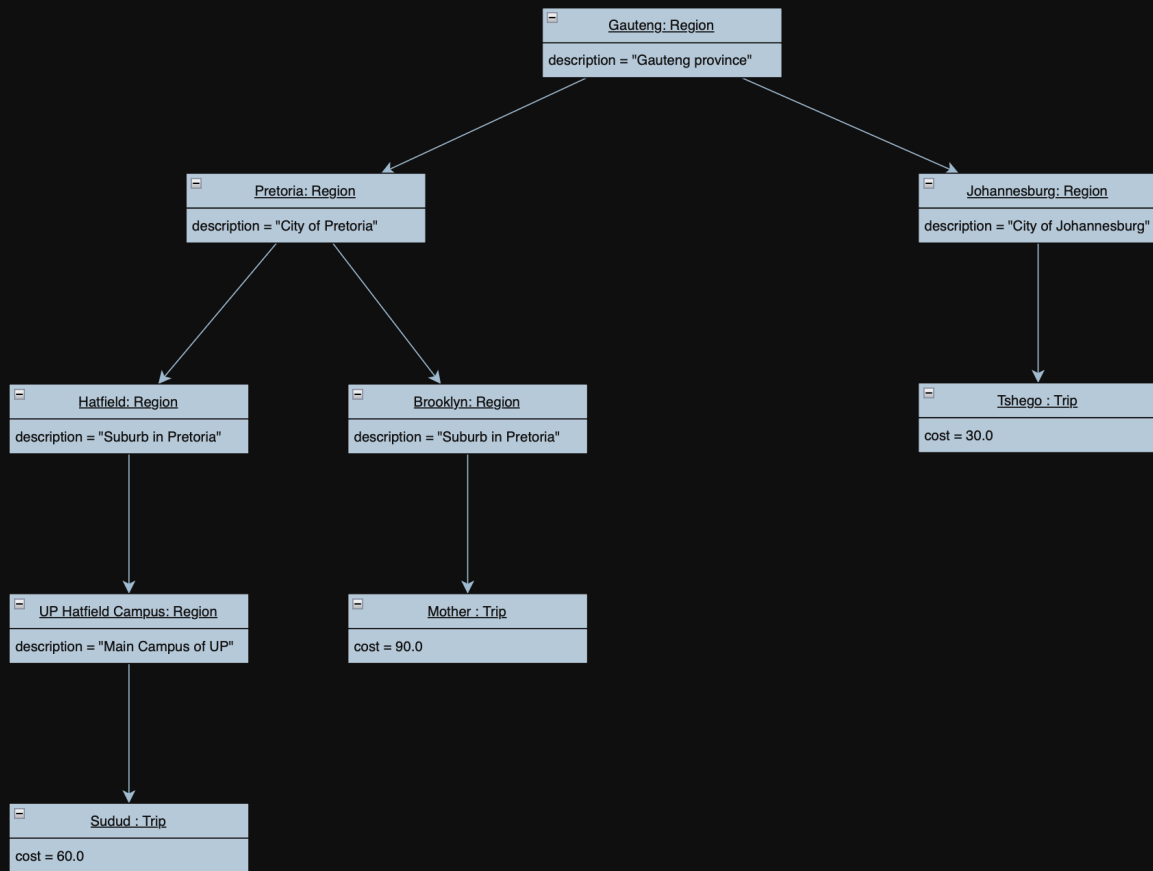
Design Rationale

The Decorator pattern class explosion by dynamically attaching features and pricing modifications to a Trip at runtime without altering the base class. The Composite pattern organises geographic areas into a tree hierarchy where Region composites and Trip leaves share a uniform component interface, allowing clients to treat individual operations and entire sub-regions identically. The State pattern encapsulates lifecycle transitions and state-dependent behaviour, eliminating fragile conditional branching (if/switch blocks) within Trip. The Iterator pattern decouples traversal logic from internal collection representations, supporting multiple specialised traversals—such as full recursive walks and in-transit filtering—without exposing internal data structures.

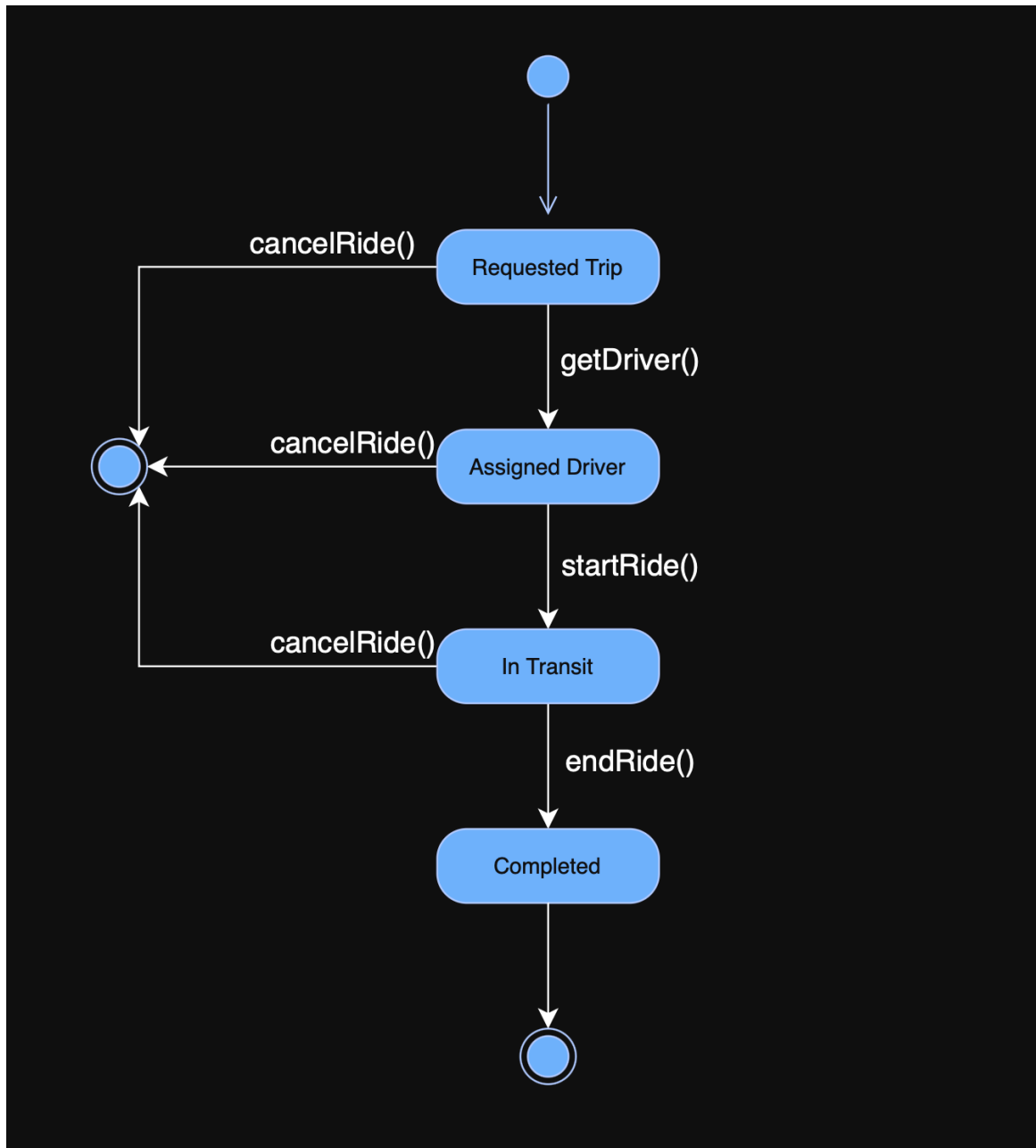
Ownership Rationale

Dynamic memory follows strict ownership boundaries. A Region composite owns its registered child components and deallocates them upon destruction. TripFeature decorators maintain ownership of their wrapped UberOperation instances, the deallocation is done in the base class Trip. The Trip context owns its active TripState instance, deleting the outgoing state upon transition and during teardown. Finally, iterators act purely as non-owning cursors, storing raw references to existing nodes without altering their lifecycles.

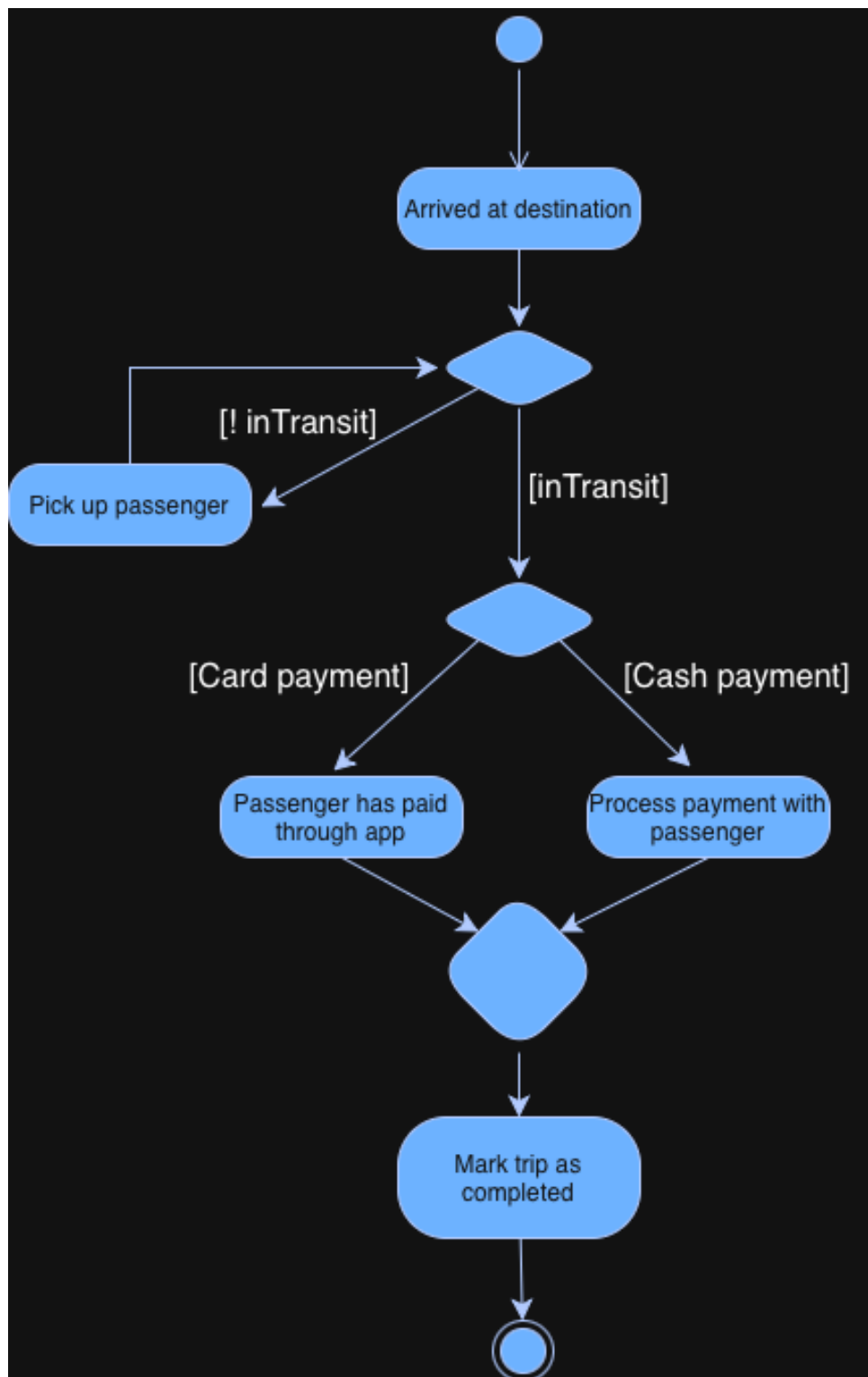
Object Diagram

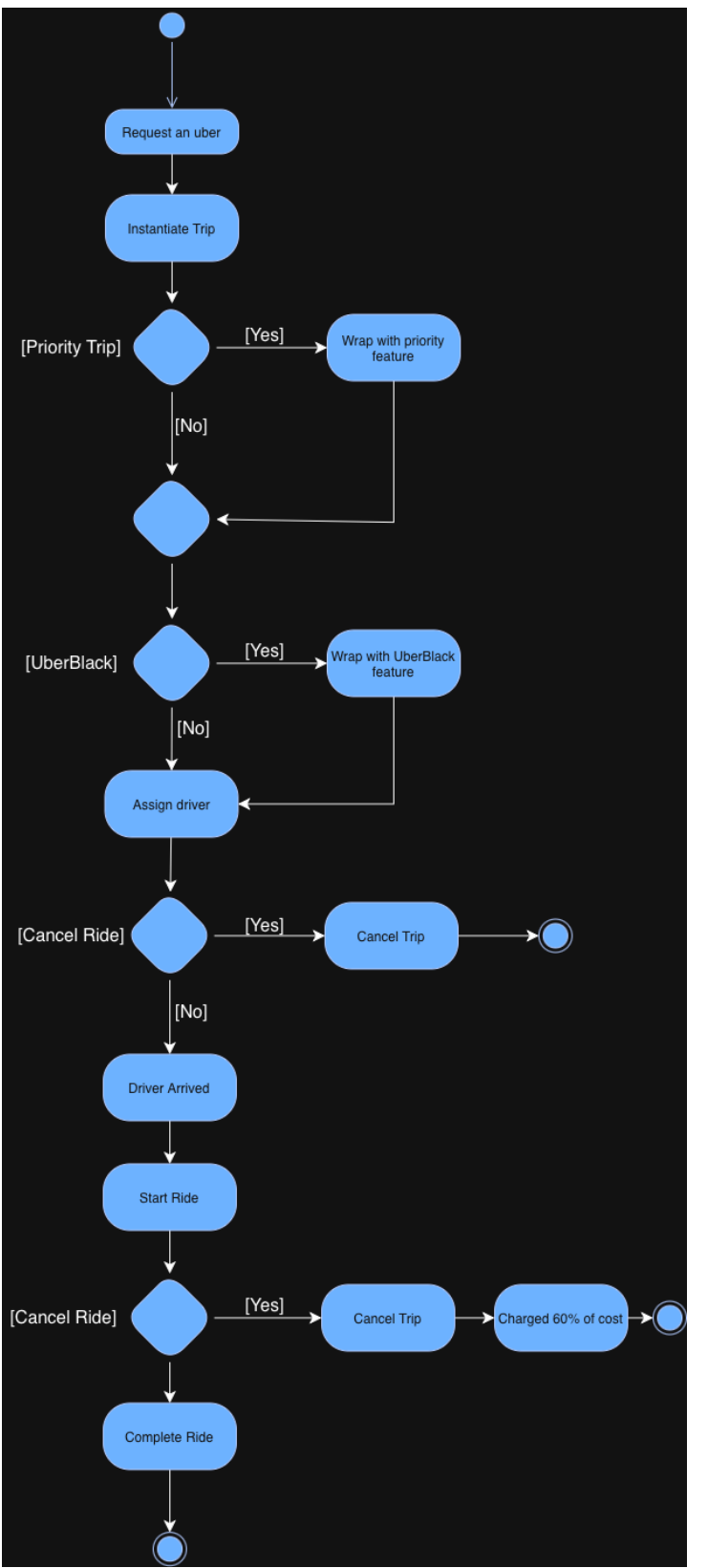
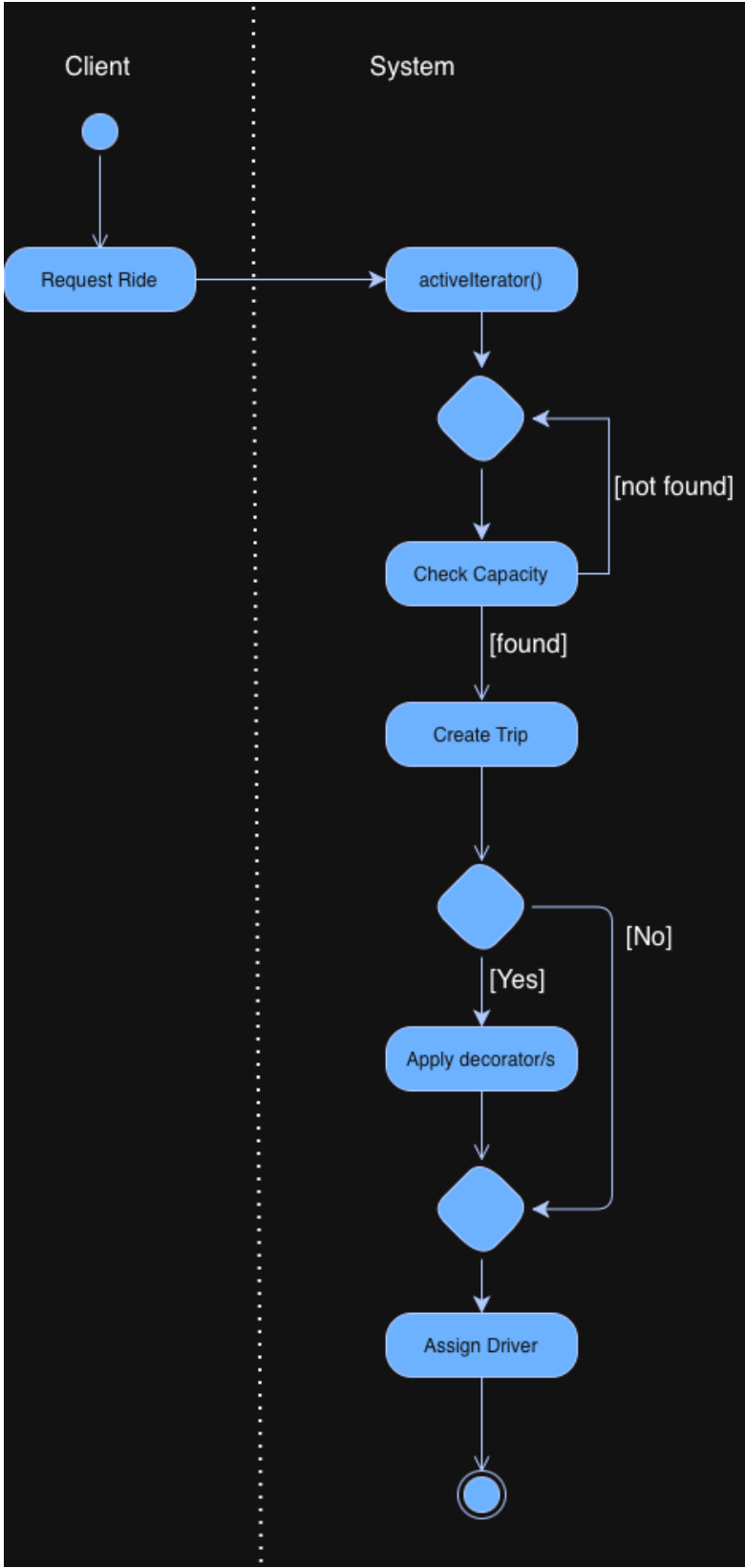


State Diagram



Activity Diagrams





Traversal-modification policy

- **Algorithm & Strategy:** Employs an iterative Depth-First Search (DFS) adhering to a Preorder traversal sequence (Node→Children).
- **State Management:** Manages traversal state using a non-recursive LIFO node stack (`std::stack<UberOperation*>`).
- **Advancement Mechanics:** Invoking `next()` pops the active node from the stack. If the element evaluates to a composite Region, its collection of child components is pushed onto the stack in reverse order.
- **Structural Preservation:** Reverse-pushing guarantees that the left-most branch is prioritised at the top of the stack, ensuring an intuitive, top-down hierarchic inspection of nested sub-regions and trips.

GDB bug investigation

```
motheo@MotheoLenovo:/mnt/c/Users/Motheo/OneDrive/Documents/COS 214/Practical 4/COS214-Prac4$ g++ -std=c++11 -Wall -Wextra -g -c InTransitState.cpp -o InTransitState.o
InTransitState.cpp: In member function 'virtual void InTransitState::endRide()':
InTransitState.cpp:18:43: error: invalid new-expression of abstract class type 'CompletedState'
   18 |         TripState *state = new CompletedState();
      |                                   ^
In file included from InTransitState.cpp:2:
CompletedState.h:9:7: note: because the following virtual functions are pure within 'CompletedState':
    9 | class CompletedState: public TripState{
      |         ^~~~~~
In file included from InTransitState.h:5,
      from InTransitState.cpp:1:
TripState.h:16:18: note: 'virtual void TripState::assignDriver(std::string)'
   16 |         virtual void assignDriver(std::string) = 0;
      |                     ^~~~~~
make: *** [makefile:54: InTransitState.o] Error 1
motheo@MotheoLenovo:/mnt/c/Users/Motheo/OneDrive/Documents/COS 214/Practical 4/COS214-Prac4$
```

Example 1:

- When compiling the application, the compiler threw an error regarding the instantiation of `CompletedState`

Motheo had made the UML diagram for our system and uploaded it to gitHub. He also had implemented .h and .cpp files. From there Sudud started making the Object diagram, State diagram and Activity diagram in accordance to the UML and the available code while also pointing out any mistakes or any ideas on how to better the system. Tshego implemented the final remaining files. Then all the work was pulled to main to make the debugging an easier effort.

Team Contribution

Team Member	Modeling & Design Artifacts	Code Implementation Deliverables	Integration & Testing Focus
Motheo	Class Diagram (UML), Repo Setup	Core interfaces, UberOperations, base .h/.cpp files	Baseline structure verification
Sudud	Object, State, and Activity Diagrams. PDF	Architectural bug fixes, Region interface overrides, main.cpp simulation	Cross-pattern verification, memory lifecycle review
Tshego	Traversal/algorithmic documentation	Concrete decorators, concrete states, iterator algorithms	Merging workflows, stack DFS testing